

THE TIME-AWARE SPLIT FOR THE bCCNN

Rolling-origin training, validation and test partitions

Study notes on the bCCNN strand of the research assignment

Prepared with the generative artificial intelligence tool Claude (Anthropic)
as study material for the BComHons research assignment
Department of Statistics and Actuarial Science
Stellenbosch University

Not for submission for assessment

October 2026

Note on the preparation of these notes

These notes were prepared with the generative artificial intelligence (AI) tool Claude (Anthropic) at the request of the student, as study material on the blended cross-classified neural network (bCCNN) strand of the research assignment. They explain the mathematics of the model, the code in the student's repository (Masters-Research-R) and the literature that the code follows.

The notes are not the student's original work and are not intended for submission for assessment. Section 2.6.3 of the Research Assignment Guide (Department of Statistics and Actuarial Science, 2026: 7-8) prohibits the use of AI tools to generate content that is submitted for assessment and requires a declaration of any use of such tools. For this reason the plagiarism declaration and the declaration on the use of artificial intelligence tools of the Department template are not included. The layout follows the technical requirements in Chapter 5 of the Guide and the Department template.

Abstract

By default the number of gradient-descent steps of the blended cross-classified neural network is not chosen on a split of the individual claims into two portfolios, but on a split of the observed claims triangle along calendar time. This report explains that time-aware split. The valuation date is moved back twice, the network is fitted on what would have been known at each earlier date, and it is scored on the calendar periods that followed; a third partition, the whole observed triangle, supplies the number of steps of the reported fit. In every partition the latest calendar periods, apart from a border of early accident and development periods, stop the training.

The report defines the partitions cell by cell, derives closed forms for their sizes, and proves the properties that the code relies on: every accident period and every development period keeps a training cell, so that the chain-ladder start can be fitted on the training cells; the latest accident and development periods stay in training; and no cell of the true lower triangle enters any partition. It then follows the fits that are run on the partitions: the early-stopping run on each partition, the out-of-time scoring of the two test partitions, the pooled test error, and the transfer of the chosen number of steps to the final fit.

The report closes with a comparison of the validation signal of this split with that of the claims split, a map of each step to the code, and a replication of the split in Python that confirms the cell counts and the properties for the configuration of the code.

Key words: Claims reserving; Neural networks; Early stopping; Rolling-origin validation; Time series cross-validation; Out-of-time testing; Chain ladder; Over-dispersed Poisson model

Table of contents

Note on the preparation of these notes.....	ii
Abstract.....	iii
List of tables.....	vi
List of figures.....	vii
List of abbreviations and/or acronyms.....	viii
CHAPTER 1 INTRODUCTION.....	1
1.1 INTRODUCTION.....	1
1.2 WHY A SPLIT ALONG TIME.....	1
1.3 SCOPE AND LIMITATIONS.....	2
1.4 HOW IT WAS CHECKED.....	3
1.5 CHAPTER OUTLINE.....	3
CHAPTER 2 CONSTRUCTION OF THE PARTITIONS.....	4
2.1 INTRODUCTION.....	4
2.2 CALENDAR PERIODS AND THE TRIANGLE OBSERVED AT AN EARLIER DATE.....	4
2.3 THE PARTITIONS.....	5
2.4 PROPERTIES OF THE SPLIT.....	7
2.5 SUMMARY.....	8
CHAPTER 3 FITTING AND SCORING ON THE PARTITIONS.....	10
3.1 INTRODUCTION.....	10
3.2 THE CCODP START ON THE TRAINING CELLS.....	10
3.3 THE EARLY-STOPPING RUN ON A PARTITION.....	10
3.4 SCORING THE TEST PARTITIONS OUT OF TIME.....	11
3.5 THE POOLED TEST ERROR.....	12
3.6 FROM THE FINAL PARTITION TO THE REPORTED MODEL.....	13
3.7 SUMMARY.....	13
CHAPTER 4 COMPARISON, IMPLEMENTATION AND VERIFICATION.....	14
4.1 INTRODUCTION.....	14

4.2	WHAT THE TWO VALIDATION SIGNALS MEASURE.....	14
4.3	WHERE EACH STEP IS IN THE CODE.....	15
4.4	NUMERICAL CHECK.....	15
4.5	SUMMARY AND CONCLUSIONS.....	16
References.....		17
APPENDIX A LIST OF SYMBOLS.....		18

List of tables

Table 2.1 Cells of the three partitions.....	7
Table 4.1 The two early-stopping signals available in config.yml.....	14
Table 4.2 Where each step of the split and of the fits is in the code.....	15
Table 4.3 Replication of <code>rolling_origin_sets()</code> for the configuration of the code.....	16
Table A.1 Symbols used in this report.....	18

List of figures

Figure 2.1 Calendar period of each cell of a 7×7 square.....	4
Figure 2.2 The three partitions of the 20×20 triangle.....	6

List of abbreviations and/or acronyms

AQ	Accident quarter
bCCNN	Blended cross-classified neural network
ccODP	Cross-classified over-dispersed Poisson (model)
DQ	Development quarter
MDN	Mixture density network
RMSprop	Root mean square propagation (optimiser)

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

This report is one of four explanatory texts on the blended cross-classified neural network (bCCNN), a network that starts from the cross-classified over-dispersed Poisson (ccODP) model, as implemented in the code of the repository. The four texts are referred to as companions. Companion I is *The mathematics behind the bCCNN: the ccODP model, the network and its chain-ladder start*; companion II is *The fitting procedure and training of the bCCNN*; companion III is *The bCCNN as a whole: interpretation, evaluation and uncertainty*; and companion IV is the present report, *The time-aware split for the bCCNN: rolling-origin training, validation and test partitions*.

The configuration file `config.yml` sets validation: `rolling_origin`, so by default the number of gradient-descent steps of the bCCNN, taken with the root mean square propagation (RMSprop) optimiser, is not chosen on the split of the claims used by Gabrielli, Richman and Wüthrich (2020), referred to as Paper C as in the code, but on a split of the observed triangle along calendar time, after Al-Mudafer, Avanzi, Taylor and Wong (2021). The report explains `rolling_origin_sets()` in `R/triangles.R` and `bccnn_ro_partition()`, `bccnn_rolling_origin()` and `bccnn_calibrate()` in `R/nn_models.R`, read against the description of the partitions and of the training in Al-Mudafer *et al.* (2021: 10, 16).

The question addressed is how the observed triangle is cut into training, validation and test cells along calendar time, what this cut guarantees for the fits of the ccODP model and of the network that are run on it, and what the resulting validation signal measures. To answer it, the report defines the split cell by cell, proves the properties the code relies on, counts the cells for the configuration of the code, and follows the three fits that are run on it: the early-stopping run on each partition, the out-of-time scoring of the test partitions, and the transfer of the chosen number of steps to the final fit. It ends with a comparison of what this split measures against what the claims split measures.

1.2 WHY A SPLIT ALONG TIME

The quantity the bCCNN predicts is the lower triangle: the payments of the calendar periods *after* the valuation date. A model is good if what it learned on calendar periods $\leq n$ carries over to calendar periods $> n$. Three ways of building a validation signal differ in how closely they imitate that question.

A random hold-out of cells. A random hold-out of cells of the same triangle measures interpolation between neighbouring cells. On a 20×20 triangle it is not even feasible for the ccODP start, which needs a cell in every accident and development period, and it is what Paper C ruled out for its 78-cell triangles (Gabrielli, Richman and Wüthrich, 2018: 11).

The 50/50 split of the claims. The split of the claims into two halves (Paper C; companion II, Section 2.3) measures how well the surface learned on one half of the portfolio fits an independent copy of the *same* upper triangle: the same accident periods, development periods and calendar periods. It is an out-of-sample signal, but not an out-of-time one.

The rolling origin. The rolling origin moves the valuation date back, fits on what would have been known then, and scores on the calendar periods that followed. This is the standard device of forecast evaluation, the 'rolling forecasting origin' (Tashman, 2000), described as time series cross-validation by Hyndman and Athanasopoulos (2021), and of reserving back-tests (removing the latest diagonals). Al-Mudafer *et al.* apply it to a 40×40 quarterly triangle: "In the first partition, the data is assumed to comprise a 30×30 triangle, which leaves the latest 10 calendar periods for the testing set. [...] The second partition works with a 36×36 triangle, leaving 4 calendar periods for testing" (Al-Mudafer *et al.*, 2021: 10), and the third partition is the whole triangle, on which the chosen model is trained. The two test partitions deliberately probe different horizons: the first "tests long-term projection", the second "recent trends" (comment in R/triangles.R).

Al-Mudafer *et al.* fit a mixture density network (MDN) to a triangle indexed by accident quarters (AQ) and development quarters (DQ). In all three partitions the validation set, which stops the training, is also chosen by time: "the validation set included the 4 latest non-testing calendar periods, excluding the first 3 accident and development periods", so that "when combined with Early Stopping [...], the MDN stops training when short term projection accuracy is maximised" (Al-Mudafer *et al.*, 2021: 10). The code scales these numbers from 40×40 quarters to 20×20 years: test horizons $(5, 2)$ for their $(10, 4)$, $v=2$ validation diagonals for their 4, and $e=2$ excluded border periods for their 3.

1.3 SCOPE AND LIMITATIONS

The report explains one component of the code, the time-aware split, and the fits that the code runs on it. The statements about Al-Mudafer *et al.* (2021) rest on the quotations in the literature notes in the repository (thesis/Hyperparameter search), which cite that paper by PDF page, and on the comments in R/triangles.R; the phrase on the replacement validation cells is taken from those comments, which give no page. The one statement attributed to Paper C is cited from the SSRN version through the same notes. These notes have not re-checked the quotations against the papers themselves.

The cell counts and the numerical properties are those of the configuration of the code: a triangle of size $n=20$, $\text{test_periods} = c(5, 2)$, $\text{vali_periods} = 2$ and $\text{exclude} = 2$. The properties are proved for general values where the derivation allows it. No network was trained for this report: the early-stopping runs, the scoring and the final fit in Chapter 3 are described from the code, and no deviances, numbers of steps or test errors are reported. The handling of periods without payments is a choice of the code that neither Al-Mudafer *et al.* nor Paper C discusses.

1.4 HOW IT WAS CHECKED

`rolling_origin_sets()` was re-implemented in Python from its definition (Section 2.3) for $n=20$, $\text{test_periods} = c(5, 2)$, $\text{vali_periods} = 2$ and $\text{exclude} = 2$, and the masks, counts, replacement cells and guarantees of Sections 2.3 and 2.4 were read off the result; Figure 2.2 is drawn from those masks. The counts agree with the closed forms (2.1), (2.2) and (2.6) derived in Chapter 2.

1.5 CHAPTER OUTLINE

Chapter 2 constructs the partitions: the calendar periods and the triangle observed at an earlier valuation date, the training, validation and test cells of each partition, and the properties of the split. Chapter 3 follows the fits run on the partitions: the ccODP start on the training cells, the early-stopping run, the out-of-time scoring of the test partitions, the pooled test error, and the transfer to the reported model. Chapter 4 compares the two validation signals available in the code, maps each step to the code, reports the numerical check, and summarises the conclusions. The symbols are listed in Appendix A.

CHAPTER 2

CONSTRUCTION OF THE PARTITIONS

2.1 INTRODUCTION

Chapter 1 argued that the validation signal should imitate the forecast of later calendar periods. This chapter turns that argument into a construction. It first defines the triangle that would have been observed at an earlier valuation date and the test cells that follow it, then the three partitions built by `rolling_origin_sets()`, and finally the properties of the split that the fits of Chapter 3 rely on. To avoid a clash with the ccODP intercept c , the valuation period of a partition (origin, `c0` in the code) is written τ .

2.2 CALENDAR PERIODS AND THE TRIANGLE OBSERVED AT AN EARLIER DATE

Cell (i, j) belongs to calendar period $k = i + j - 1$; the cells of one calendar period form a diagonal of the square. At the valuation date n the observed triangle is $D = \{(i, j) : k \leq n\}$. Figure 2.1 shows the calendar period $k = i + j - 1$ of each cell of a 7×7 square; moving the valuation date back from 7 to τ removes the diagonals $k > \tau$.

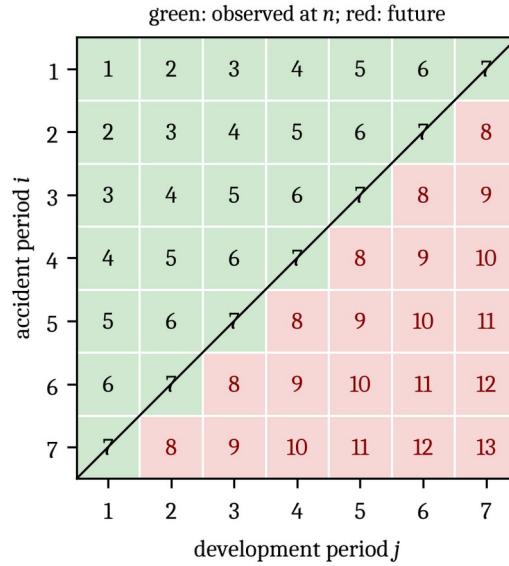


Figure 2.1 Calendar period of each cell of a 7×7 square

A reserving actuary working at the earlier valuation date $\tau < n$ would have seen accident periods $1, \dots, \tau$, development periods $1, \dots, \tau$, and only the cells with $k \leq \tau$:

$$D_\tau = \{(i, j) : 1 \leq i, j \leq \tau, i + j - 1 \leq \tau\}, |D_\tau| = \frac{\tau(\tau+1)}{2}. \quad (2.1)$$

Any model fitted on D_τ can forecast only cells of the $\tau \times \tau$ square: the ccODP has effects for the periods $\leq \tau$ only, and so has the bCCNN, whose embeddings have τ categories. Of those cells, the ones that lie after τ but were observed by n are the test cells of the partition,

$$\begin{aligned} T_\tau &= \{(i, j) : 1 \leq i, j \leq \tau, \tau < i + j - 1 \leq n\}, \\ |T_\tau| &= \sum_{k=\tau+1}^n (2\tau - k)(n \leq 2\tau - 1), \end{aligned} \quad (2.2)$$

because diagonal $k \geq \tau$ of a $\tau \times \tau$ square holds the cells $i = k + 1 - \tau, \dots, \tau$, that is $2\tau - k$ of them. For $\tau = 15$: $14 + 13 + 12 + 11 + 10 = 60$ cells; for $\tau = 18$: $17 + 16 = 33$. The cells of D outside the $\tau \times \tau$ square (accident periods $> \tau$, or development periods $> \tau$) are observed but unused by that partition, and the true lower triangle F is used by no partition at all.

2.3 THE PARTITIONS

`rolling_origin_sets(m, test_periods = c(5, 2), vali_periods = 2, exclude = 2)` first cuts its input to the observed triangle, `y <- upper(m)`, so nothing below the diagonal $k = n$ can enter. It then builds one partition per valuation period

$$\tau \in \{n - t_1, n - t_2, n\} = \{15, 18, 20\}, \quad (2.3)$$

test partitions first, the final one last. For each τ it takes the $\tau \times \tau$ square, computes k for every cell, and defines the validation, training and test cells as follows.

Validation cells. The validation cells are the latest v diagonals of D_τ , without the first e accident periods and the first e development periods,

$$V_\tau^{diag} = \{(i, j) \in D_\tau : k > \tau - v, i > e, j > e\}, \quad (2.4)$$

plus $K = v(e - 1)$ replacement cells at the excluded early development periods (Al-Mudafer *et al.* (2021), as quoted in the comments of `R/triangles.R`: the "DQ2 and DQ3 validation points ... taken evenly from earlier AQs"). With $lo = e + 1$ and $hi = \tau - v - e + 1$, the replacement cells sit at development periods $e, e - 1, \dots, 2$ (cycled K times; with $e = 2$ both at development period 2) and at the accident periods at the centres of K equal bins of $[lo, hi]$,

$$\begin{aligned} E_\tau &= \{(r_m, d_m) : m = 1, \dots, K\}, \\ r_m &= \text{round}\left(lo + (hi - lo) \frac{m - 1/2}{K}\right), d_m = e - ((m - 1) \bmod (e - 1)), \\ V_\tau &= V_\tau^{diag} \cup E_\tau. \end{aligned} \quad (2.5)$$

R's `round()` rounds halves to even, as the Python replication does. For the configuration of the code $K=2$, $lo=3$, and $hi=12, 15, 17$ for $\tau=15, 18, 20$, giving the replacement cells $(5, 2), (10, 2); (6, 2), (12, 2); (6, 2), (14, 2)$.

Training cells. The training cells are everything observed at τ that is not validation: $U_\tau = D_\tau \setminus V_\tau$.

Test cells. The test cells are T_τ of (2.2) for the test partitions; the final partition ($\tau=n$) has none.

Each partition is returned as a list with the $\tau \times \tau$ matrix of payments observed at τ (y, NA after τ), the logical masks `train` and `vali`, the matrix `test` (payments on T_τ , NA elsewhere, NULL for the final partition), a role matrix for the plots, `origin = \tau`, `final`, and `n`.

Figure 2.2 shows the three partitions of the 20×20 triangle for `test_periods = c(5, 2)`, `vali_periods = 2` and `exclude = 2`. The training cells are light green. The validation cells are dark green: the last two diagonals of each partition minus the first two accident and development periods, plus two cells at development period 2. The test cells are red: the calendar years after τ up to 20, inside the $\tau \times \tau$ square. Hatched cells were observed at year 20 but lie outside the partition's square; grey cells form the true lower triangle, used by no partition.

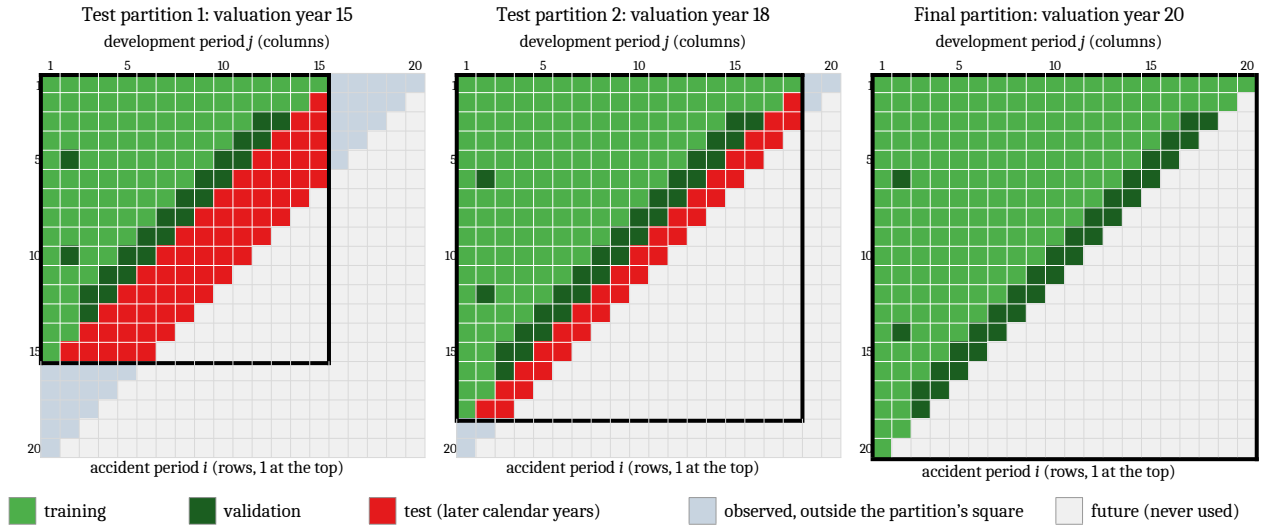


Figure 2.2 The three partitions of the 20×20 triangle

Table 2.1 counts the cells of the three partitions in the Python replication of `rolling_origin_sets()`. The counts are those before the masking of periods without payments (Section 3.3). The test partitions together hold 93 test cells.

Table 2.1 Cells of the three partitions

Partition	τ	Square	Observed $ D_\tau $	Training $ U_\tau $	Validation $ V_\tau $ (diagonals + E_τ)	Validation cells by calendar period	Test $ T_\tau $ (calendar periods)
Test 1	15	15×15	120	97	23 (21 + 2)	$k = 14: 10$, $k = 15: 11$, $k = 6: 1$, $k = 11: 1$	60 ($k = 16, \dots, 20$)
Test 2	18	18×18	171	142	29 (27 + 2)	$k = 17: 13$, $k = 18: 14$, $k = 7: 1$, $k = 13: 1$	33 ($k = 19, 20$)
Final	20	20×20	210	177	33 (31 + 2)	$k = 19: 15$, $k = 20: 16$, $k = 7: 1$, $k = 15: 1$	None

2.4 PROPERTIES OF THE SPLIT

The construction of Section 2.3 has seven properties, labelled (a) to (g), on which the code and the fits of Chapter 3 rely.

(a) *Every accident period and every development period of D_τ has a training cell.* Accident periods $i \leq e$: none of their cells is on a validation diagonal (2.4), and the replacement rows satisfy $r_m \geq l_0 = e + 1$, so the whole row is training. Accident periods $i > e$: the cell $(i, 1)$ has $j = 1 \leq e$, so it is excluded from (2.4), and the replacement cells have $d_m \geq 2$, so $(i, 1)$ is training. Development periods $j \leq e$: the cell $(1, j)$ is in accident period $1 \leq e$, hence training. Development periods $j > e$: the cell $(1, j)$ is in accident period 1, hence training. In short, the first e rows and the first column are entirely training, and that is enough. This is the guarantee `fit_odp_glm(cells = part$train)` needs ("every accident and development period needs at least one cell to fit on") and the reason the comment in `R/triangles.R` says the exclusions make the ccODP start fittable on the training cells. The Python replication confirms it for all three partitions.

(b) *Counts.* Diagonal $k \leq \tau$ of D_τ has k cells, $i = 1, \dots, k$; removing $i \leq e$ and $j = k + 1 - i \leq e$ removes $2e$ of them. Hence

$$\begin{aligned}
 |V_\tau| &= \sum_{k=\tau-v+1}^{\tau} (k - 2e) + v(e - 1) \\
 &= v(\tau - 2e) - \frac{v(v-1)}{2} + v(e - 1), \\
 |U_\tau| &= \frac{\tau(\tau+1)}{2} - |V_\tau|,
 \end{aligned} \tag{2.6}$$

which gives 23, 29, 33 validation cells and 97, 142, 177 training cells for $\tau = 15, 18, 20$, as in Table 2.1. The validation share of the observed cells is 19.2 %, 17.0 %, 15.7 %.

(c) *The latest accident periods and the latest development periods stay in training.* Accident period τ has the single cell $(\tau, 1)$, and accident period $\tau - 1$ the cells $(\tau - 1, 1), (\tau - 1, 2)$: all have $j \leq e$ and are training. Likewise development periods τ and $\tau - 1$ consist of $(1, \tau)$ and $(1, \tau - 1), (2, \tau - 1)$, all in accident periods $\leq e$, all training. So the network keeps learning the level of the most recent accident periods and the tail of the development pattern, while being stopped on the most recent diagonals of the periods in between. This is the point of the exclusion of Al-Mudafer *et al.*: without it, the latest accident period would have no training cell at all (its only cell is on the last diagonal), and a cross-classified model could not even be started.

(d) *The replacement cells lie off the validation diagonals.* A replacement cell has calendar period $r_m + d_m - 1 \leq h i + e - 1 = \tau - v$, so it is earlier than every diagonal of (2.4): it adds validation information about the early development periods (which (2.4) removed) without touching the latest diagonals, and it is spread over the accident periods $[l o, h i]$ so as not to cluster. For the final partition the two cells are $(6, 2)$ and $(14, 2)$, calendar periods 7 and 15.

(e) *No leakage.* The input is cut to D before anything else, so the true lower triangle is in no partition. Within a partition, fitting and early stopping read only cells with $k \leq \tau$; the cells with $\tau < k \leq n$ are read by the scoring alone (Section 3.4), and only for the test partitions. The final partition's run tracks the true lower triangle (truth) for a plot, after the fact, without using it.

(f) *The partitions are nested in time, not in cells.* $D_{15} \subset D_{18} \subset D_{20} = D$, but the validation sets are not nested: the last two diagonals of the 15×15 triangle are training cells of the 18×18 one. Each partition is a complete, self-contained imitation of the reserving problem at its own valuation date.

(g) *Scaling.* The values $(10, 4)/4/3$ of Al-Mudafer *et al.* on 40×40 quarters hold out 25 % and 10 % of the calendar periods for testing and 10 % for validation; the values $(5, 2)/2/2$ of the code on 20×20 years keep the same proportions of calendar periods (25 %, 10 %, 10 %) and shrink the border from three periods to two. The function stops if a partition would be too small for the requested validation and border periods ($\tau - v < 2e + 1$).

2.5 SUMMARY

The rolling origin cuts the observed triangle into three partitions with valuation periods 15, 18 and 20. Each partition consists of the triangle that would have been observed at its valuation date; its latest two diagonals, without the first two accident and development periods, plus two replacement cells at development period 2, are the validation cells, and the rest are the training cells. The two test partitions are scored on the 60 and 33 cells observed after their valuation dates inside their squares. The

construction guarantees a training cell in every accident and development period, keeps the latest accident and development periods in training, and keeps the true lower triangle out of every partition. Chapter 3 follows the fits that are run on these partitions.

CHAPTER 3

FITTING AND SCORING ON THE PARTITIONS

3.1 INTRODUCTION

Chapter 2 constructed the partitions and their properties. This chapter follows what the code does with them: the ccODP start fitted on the training cells of each partition, the early-stopping run on each partition, the out-of-time scoring of the test partitions, the pooling of their scores into one test error per model, and the transfer of the number of steps chosen on the final partition to the reported model.

3.2 THE CCODP START ON THE TRAINING CELLS

For each partition `bccnn_ro_partition()` fits the ccODP on the training cells only, `fit_odp_glm(part$y, cells = part$train)`, giving $\hat{\vartheta}(U_\tau) = (\hat{c}, \hat{\alpha}_1, \dots, \hat{\alpha}_\tau, \hat{\beta}_1, \dots, \hat{\beta}_\tau)$ with $2\tau - 1$ parameters. The maximum likelihood equations (2.9) of companion I then hold over the training cells: the fitted row and column totals over U_τ equal the observed ones over U_τ . This has two consequences.

The start is not the chain ladder of the partition. The start of the partition's network is *not* the chain ladder of D_τ , which would use all of D_τ including the validation diagonals. It is the ccODP of an irregular set of cells. The chain ladder at τ , $\hat{\vartheta}(D_\tau)$, is fitted separately for the scoring (`cl <- fit_odp_glm(part$y)`), so that the test partitions report two chain-ladder-type baselines: the chain ladder as it would have been reserved at τ (`test_ccODP`), and the training-cell ccODP that the network started from (`test_ccODP_train`, which is also the `test_pred` of the partition's history at step 0).

The gradient at the start. Property 2 of companion I (Section 4.5: zero gradient in c and w at the start, only B moves at step 1) holds on the training cells, because it only needs the marginal-total equations on the cells the loss is computed on.

Unlike the claims split, the training cells carry the full volume of the portfolio, so $\hat{c}$ is on the scale of the whole triangle; what is removed is cells, not claims.

3.3 THE EARLY-STOPPING RUN ON A PARTITION

Each partition then receives the run of companion II (Section 4.2) with its own data: from $\theta_0(\hat{\vartheta}(U_\tau))$, $T_{max} = 1\,000$ full-batch RMSprop steps on the $|U_\tau|$ training cells with dropout, recording after every step the deviance without dropout on the training cells, on the validation cells,

$$D_{V,\tau}(t) = D(y, \mu(\theta_t); V_\tau), t = 0, \dots, T_{max}, \quad (3.1)$$

on the test cells T_τ (test partitions; column test), and on the true lower triangle (final partition; column truth), together with the predicted sums over each of those sets ($*_{\text{pred}}$). The step with the lowest validation deviance,

$$t_\tau^* = \arg \min_{0 \leq t \leq T_{\max}} D_{V,\tau}(t), \quad (3.2)$$

the decreases $\text{loss_decrease}(h, \text{best})$ and the ccODP are kept. Because the partition's batch is its $|U_\tau|$ training cells, each partition is a separate Keras model built with the same seed: the same initial hidden-layer weights, but, since the batch sizes differ, different dropout mask sequences. For the final partition the whole path of mean squares is kept ($\text{keep_mu} = \text{TRUE}$ when $\text{final_fit} = \text{"partition"}$), so that its early-stopped network can be reported without retraining (Section 3.6).

Periods without payments. Property (a) guarantees each period a training *cell*, not a training *payment*. If all training cells of a development period hold zero, for instance a late development period whose only payment of the partition sits on a validation diagonal, the ccODP on the training cells gives that period an effect of about -30 and fitted means of order e^{-30} (companion I, Section 2.10). A validation cell with a positive payment there would add about $30 y$ to $D_{V,\tau}(t)$ at every step, and could decide t_τ^* by itself. `bccnn_ro_partition()` therefore applies `mask_zero_periods()` with the training-cell ccODP: those validation cells are left out of (3.1), as they are under the claims split, and V_τ in (3.1) and (3.2) means the validation cells actually scored. Neither Al-Mudafer *et al.* nor Paper C nor Härkönen (2021) discusses this case.

3.4 SCORING THE TEST PARTITIONS OUT OF TIME

A test partition answers the following question: had the procedure been applied at valuation date τ , how well would it have forecast the next $n - \tau$ calendar periods? `bccnn_ro_partition()` scores two things on T_τ , with `final_fit` deciding which network stands for the procedure.

Refit. With `final_fit = "refit"` (the default, the final fit of Paper C), the procedure *as it is reported* is replayed at τ . The chain ladder of D_τ gives the start, and the network is trained on all of D_τ for exactly t_τ^* steps,

$$\begin{aligned} \theta_\tau^{\text{refit}} &= T_{t_\tau^*}^s \left(\theta_0 \left(\hat{\vartheta}(D_\tau) \right), D_\tau \right), \\ \text{test loss} &= D \left(y, \mu \left(\theta_\tau^{\text{refit}} \right); T_\tau \right), \\ \text{test prediction} &= \sum_{T_\tau} \mu_{i,j} \left(\theta_\tau^{\text{refit}} \right). \end{aligned} \quad (3.3)$$

This is a genuine out-of-time back-test of the whole chain "choose t^* on the latest diagonals, refit on everything, forecast": every number in (3.3) could have been produced at τ .

Partition. With `final_fit = "partition"` (Al-Mudafer *et al.*), the early-stopped network of the partition itself, $\theta_{t_\tau^*}$ trained on U_τ , is scored: the test loss and test prediction are read from the history at step t_τ^* .

Test cells receive the same treatment as validation cells, with a different reference. A test cell is left out if its accident or development period has no payments in *any* cell observed at τ , that is, if the chain ladder at τ flags it: no model fitted at τ could forecast it. A test cell in a period that has payments only in the validation cells stays in. The chain ladder at τ does know that period, so a variant that loses it, such as the partition network started from the training-cell ccODP, is scored for the loss. All three models are scored on the same test cells, and `n_test` counts them.

Next to the bCCNN, the partition records the actual test sum $\sum_{T_\tau} y_{i,j}$, the chain ladder at τ (its test sum and test deviance) and the training-cell ccODP (its test sum and test deviance from the history at step 0). The comment in the code names which baseline is like-for-like with which variant: the chain ladder at τ for refit, the training-cell ccODP for partition.

3.5 THE POOLED TEST ERROR

`bccnn_rolling_origin()` pools the test partitions into one number per model, the test deviance per test cell of equation (4.4) of Al-Mudafer *et al.*,

$$\begin{aligned} \dot{D}^M &= \frac{\sum_{\tau \in \{15, 18\}} D(y, \mu_\tau^M; T_\tau)}{\sum_{\tau \in \{15, 18\}} |T_\tau|}, \\ M &\in \{ \text{ccODP fitted at } \tau, \text{ccODP on } U_\tau, \text{bCCNN} \}, \end{aligned} \quad (3.4)$$

with $60 + 33 = 93$ test cells in the denominator (fewer if a test partition has a period without payments by τ , Section 3.4). It is cell-weighted, so the long-horizon partition ($\tau = 15$, five calendar years ahead) weighs almost twice the short-horizon one; equation (4.4) of Al-Mudafer *et al.* averages the negative log-likelihood of their density forecasts in the same way, and the code uses the deviance because the bCCNN is a mean model. `bccnn_tables()` prints the per-partition losses, their per-cell versions, and the pooled row labelled "test error (4.4)".

What the test error is for. In Al-Mudafer *et al.* the test error is the objective of the hyperparameter search ("The MDN's architecture was selected using an algorithm that successively optimised one hyperparameter at a time" (Al-Mudafer *et al.*, 2021: 11)), averaged over several seeds to damp the initialisation (Al-Mudafer *et al.*, 2021: 16). In the code the architecture is fixed by `config.yml`, so $\dot{D}$ is *reported*, not optimised: it shows whether the bCCNN procedure beat the chain ladder out of time at two earlier valuation dates, in the same units as the in-sample and out-of-sample deviances of the final fit, before the true lower triangle is looked at. It is a single realisation per seed (the comment in `bccnn_rolling_origin()`:

"results depend on the seed"); comparing designs by it would need the seed averaging that Al-Mudafer *et al.* perform.

3.6 FROM THE FINAL PARTITION TO THE REPORTED MODEL

The number of steps of the final fit is that of the final partition,

$$t^* = t_n^*, \text{ chosen on } V_{20}: \quad (3.5)$$

the diagonals $k=19, 20$ minus the border, plus $(6, 2), (14, 2)$.

`bccnn_calibrate()` then proceeds exactly as under the claims split (companion II, Sections 4.3 and 5.2): under refit, the chain ladder of D is fitted and the network is trained on all 210 cells for t^* steps from it, $\hat{\theta} = T_{t^*}^s(\theta_0(\hat{\vartheta}(D)), D)$; under partition, the mean square of the final partition's network at step t^* is taken from the stored path, with no second training. The decreases of Paper C's Table 3 and the bCCNN dispersion $\hat{\phi}^{bCCNN} = \hat{\phi}_D(1 - \max(0, \Delta_V))$ are computed from the final partition's curve, so Δ_V is now the relative decrease of the deviance on the latest two diagonals rather than on an independent half of the portfolio.

The transfer of t^* rests on the same assumption as under the claims split, that the loss surface of the full triangle is close to the one the run explored, and it is a milder assumption here in one respect and a stronger one in another. It is milder because the training cells of the final partition are 177 of the 210 cells of the same portfolio, with the same intercept scale, rather than half the claims. It is stronger because the 33 cells that the refit adds are the latest diagonals, the cells nearest to the forecast region, and the refit trains on them for t^* steps without any check on them.

3.7 SUMMARY

On each partition the ccODP is fitted on the training cells only, which keeps the marginal-total equations and hence the behaviour of the network at the start, but makes the start differ from the chain ladder at the valuation date. The early-stopping run records the validation deviance after every step and chooses the step with the lowest value; validation and test cells in periods without payments are masked so that a single cell cannot decide the result. The two test partitions score the procedure out of time, either as a refit at the earlier valuation date or as the early-stopped network of the partition, and their deviances are pooled into a cell-weighted test error per model, which the code reports rather than optimises. The number of steps of the reported model is the one chosen on the final partition. Chapter 4 compares this validation signal with the claims split and verifies the construction numerically.

CHAPTER 4

COMPARISON, IMPLEMENTATION AND VERIFICATION

4.1 INTRODUCTION

Chapters 2 and 3 described the rolling origin on its own. This chapter places it next to the claims split of Paper C and Härkönen (2021), maps each step of the construction and of the fits to the code, and reports the Python replication that checks the cell counts and properties of Chapter 2.

4.2 WHAT THE TWO VALIDATION SIGNALS MEASURE

Table 4.1 sets the two early-stopping signals available in config.yml side by side.

Table 4.1 The two early-stopping signals available in config.yml

Aspect	Claims split (Paper C, Härkönen)	Rolling origin (Al-Mudafer <i>et al.</i>)
What is split	The individual claims, into two portfolios	The cells of the observed triangle, by calendar period
Training data of the run	The training half's upper triangle, 210 cells, half the volume	The training cells U_t : 177 of 210 cells (final partition), full volume
Validation data	The validation half's upper triangle, 210 cells (minus zero periods): the same (i, j) pairs as training	The latest two diagonals minus the border, plus two early cells: 33 cells, (i, j) pairs the run never trained on
What D_v estimates	Out-of-sample fit at the same accident, development and calendar periods: interpolation in an independent copy	Out-of-sample fit one and two calendar periods ahead of the training cells: short-term extrapolation
Start of the run	ccODP of the training half (intercept lower by about $\log 2$)	ccODP of the training cells (not the chain ladder of the partition)
Out-of-time check before the truth is used	None	The test error (3.4) at two earlier valuation dates
Randomness in the split	The allocation of the claims (results change when the halves are swapped; Härkönen, 2021: 33-34)	None: the split is deterministic given the configuration
What is transferred to the final fit	t^*	t^* of the final partition (or the partition's network itself)
Cost	One ccODP, one run	Three ccODPs and three runs, plus two refits for the scoring

Neither signal measures what the back-test measures, the deviance on the true lower triangle, which is 190 cells up to 19 calendar periods ahead; test partition 1 of the rolling origin (five periods ahead) is the closest a procedure can come without looking at the truth. For a short-tailed annual triangle, where most of the reserve sits in the first few development periods of the latest accident periods, 'short-term projection accuracy' is also most of the reserve, which is the case for stopping on the latest diagonals.

4.3 WHERE EACH STEP IS IN THE CODE

Table 4.2 gives, for each step of the construction and of the fits, the function and the lines of code that carry it out.

Table 4.2 Where each step of the split and of the fits is in the code

Step	Where in the code
Cut to the observed triangle; calendar periods (2.1)	<code>rolling_origin_sets(): y <- upper(unname(as.matrix(m))), cal <- row(ys) + col(ys) - 1L, known <- cal <= c0</code>
Valuation periods (2.3)	<code>origins <- c(n - sort(test_periods, decreasing = TRUE), n)</code>
Validation diagonals (2.4)	<code>vali <- known & cal > c0 - vali_periods & row(ys) > exclude & col(ys) > exclude</code>
Replacement cells (2.5)	<code>k <- vali_periods * (exclude - 1L), lo <- exclude + 1, hi <- c0 - vali_periods - exclude + 1, rows <- round(lo + (hi - lo) * (seq_len(k) - 0.5) / k), devs <- rep(exclude:2, length.out = k), vali[cbind(rows, devs)] <- TRUE</code>
Training and test cells, (2.2)	<code>train <- known & !vali; test <- if (c0 < n) ifelse(cal > c0, ys, NA) else NULL (the NAs of ys beyond $k = n$ keep the test inside D)</code>
Size check, property (g)	<code>if (any(origins - vali_periods < 2 * exclude + 1)) stop(...)</code>
ccODP on the training cells, Section 3.2	<code>bccnn_ro_partition(): odp <- fit_odp_glm(part\$y, scale, phi, cells = part\$train); cl <- fit_odp_glm(part\$y, ...) for the chain ladder at τ</code>
Masking of periods without payments, Sections 3.3 and 3.4	<code>bccnn_ro_partition(): cl <- fit_odp_glm(part\$y, ...) first, then track\$vali <- mask_zero_periods(ifelse(part\$vali, odp\$y, NA), odp), track\$test <- mask_zero_periods(part\$test / scale, cl); n_vali, n_test count the scored cells</code>
The run (3.1) and (3.2)	<code>nn <- fit_bccnn(odp, epochs = max_epochs, track = track, keep_mu = keep_mu), best <- h\$epoch[which.min(h\$vali)]</code>
Scoring (3.3)	<code>if (final_fit == "refit") rf <- fit_bccnn(cl, epochs = best, track = list(test = track\$test)) else at("test", best); the out\$test vector</code>
Test error (3.4)	<code>bccnn_rolling_origin(): test_error <- c(ccODP = ..., ccODP_train = ..., bCCNN = ...) / sum(tst\$n_test); bccnn_tables() row "test error (4.4)"</code>
Final partition to the final fit (3.5)	<code>bccnn_calibrate(): val <- ro\$final, steps <- val\$best_epoch, then fit_bccnn(odp, epochs = steps, ...) or val\$mu_path[[steps + 1]]</code>
Figures	<code>plot_partition(part) in R/plots.R (roles as tiles), called per partition by the script</code>

4.4 NUMERICAL CHECK

The Python replication (`rolling_origin_check.py`, kept with these explainers) reproduces Table 2.1 and the properties of Section 2.4 for the configuration of the code, that is, `test_periods = c(5, 2)`, `vali_periods = 2`,

exclude = 2 and $n = 20$. Table 4.3 lists the results; the three entries in each row refer to the partitions with $\tau = 15, 18, 20$.

Table 4.3 Replication of `rolling_origin_sets()` for the configuration of the code

Check	Result
Valuation periods (2.3)	15, 18, 20
Cells observed at τ , (2.1)	120, 171, 210
Validation cells, (2.6): on the diagonals + replacements	21+2, 27+2, 31+2
Replacement cells (2.5): (lo, hi) and the cells	$(3, 12): (5, 2), (10, 2); (3, 15): (6, 2), (12, 2);$ $(3, 17): (6, 2), (14, 2);$ calendar periods 6, 11; 7, 13; 7, 15, all $\leq \tau - v$
Training cells	97, 142, 177; in the last two accident periods of each partition: 1 and 2 cells, all training
Every accident period and every development period has a training cell, (a)	True, true, true
Test cells (2.2) and their calendar periods	60 ($k = 16$ to 20), 33 ($k = 19, 20$), none; 93 in total
Validation share of the observed cells	19.2%, 17.0%, 15.7%

4.5 SUMMARY AND CONCLUSIONS

The rolling origin replaces the claims split as the default early-stopping signal of the bCCNN. Where the claims split measures interpolation in an independent copy of the same upper triangle, the rolling origin stops the training on the one and two calendar periods that follow the training cells, and scores the whole procedure out of time at two earlier valuation dates before the true lower triangle is used. The split is deterministic given the configuration, and it costs three ccODP fits and three runs, plus two refits for the scoring.

The construction in `rolling_origin_sets()` follows Al-Mudafer *et al.* (2021) with their proportions of calendar periods scaled from quarters to years. Its exclusion of the first accident and development periods from the validation diagonals guarantees that every period keeps a training cell, so that the ccODP start can be fitted on the training cells, and its replacement cells restore validation information at the early development periods without touching the latest diagonals. The Python replication confirms the cell counts, the replacement cells and these guarantees for the configuration of the code.

Two limitations remain. The transfer of the number of steps to the final fit trains on the latest diagonals without any check on them, and the pooled test error is a single realisation per seed, so comparing designs by it would require averaging over seeds as Al-Mudafer *et al.* do. Neither validation signal measures the deviance on the true lower triangle; the long-horizon test partition is the closest a procedure can come without looking at the truth.

References

- Al-Mudafer, M.T., Avanzi, B., Taylor, G. & Wong, B. 2021. Stochastic loss reserving with mixture density neural networks. arXiv preprint 2108.07924.
- Department of Statistics and Actuarial Science. 2026. *Research assignment guide*. Last updated 28 January 2026. Stellenbosch: Stellenbosch University.
- Gabrielli, A., Richman, R. & Wüthrich, M.V. 2020. Neural network embedding of the over-dispersed Poisson reserving model. *Scandinavian Actuarial Journal*, 2020(1):1-29.
- Gabrielli, A., Richman, R. & Wüthrich, M.V. 2018. Neural network embedding of the over-dispersed Poisson reserving model. SSRN working paper 3288454, version of 21 November 2018.
- Härkönen, V. 2021. On claims reserving with machine learning techniques. Unpublished master's thesis. Stockholm: Stockholm University.
- Hyndman, R.J. & Athanasopoulos, G. 2021. *Forecasting: principles and practice*. 3rd edition. Melbourne: OTexts.
- Tashman, L.J. 2000. Out-of-sample tests of forecasting accuracy: an analysis and review. *International Journal of Forecasting*, 16(4):437-450.

APPENDIX A

LIST OF SYMBOLS

The symbols in the upper block are those of companions I and II, and those in the lower block are new in this report; the third column gives the section where each symbol is introduced.

Table A.1 Symbols used in this report

Symbol	Meaning	Section
n	Size of the triangle, $n = 20$	2.2
i, j, k	Accident period, development period, calendar period $k = i + j - 1$	2.2
D	The observed triangle at the valuation date n (sets\$upper), 210 cells	2.2
F	The future cells, $k > n$; never used here	2.2
$D(y, \mu; S)$	Poisson deviance on the cells S	3.3
$\theta_0(\hat{\vartheta}), T_t^s(\theta, S)$	The ccODP start built from a fit $\hat{\vartheta}$; t RMSprop steps with seed s on the cells S (companion II, Section 4.4)	3.2 to 3.4
T_{max}	Length of each early-stopping run, 1 000	3.3
	New in this report	
τ	Valuation period of a partition: $\tau \in \{15, 18, 20\}$	2.3
t_1, t_2	Calendar periods held out by the test partitions, test_periods $= (5, 2)$	2.3
v	Validation calendar periods per partition, vali_periods = 2	2.3
e	First accident and development periods excluded from the validation diagonals, exclude = 2	2.3
D_τ	The triangle observed at τ : the cells of the $\tau \times \tau$ square with $k \leq \tau$	2.2
T_τ	Test cells: cells of the $\tau \times \tau$ square with $\tau < k \leq n$	2.2
V_τ, U_τ	Validation and training cells of the partition, $U_\tau = D_\tau \setminus V_\tau$	2.3
E_τ	The $K = v(e - 1)$ replacement validation cells at early development periods	2.3

Symbol	Meaning	Section
$l o, h i$	The range of accident periods the replacement cells are spread over	2.3
$\hat{\vartheta}(U_{\tau}), \hat{\vartheta}(D_{\tau})$	ccODP fitted on the training cells; on all cells observed at τ (the chain ladder at τ)	3.2
$D_{v,\tau}(t), t_{\tau}^*$	Validation deviance of the partition after t steps; its minimiser	3.3
$\acute{D}^M$	Test error of model M over the test partitions, equation (4.4) of Al-Mudafer <i>et al.</i> (2021)	3.5
t^*	The number of steps of the final fit, $t^* = t_n^*$	3.6